Integración de Servicios Cloud GCP

Docker: tu propia imagen, puertos, variables y Compose

Semana 14 - Sesión 28

Rel Guzman



Revisión: lo que vimos en la Sesión 27

Desplegar y dependencias

Desplegar es dejar una app funcionando en un servidor. Una dependencia es lo que la app necesita y no es su código: Python, Flask.

Imagen y contenedor

La imagen es la app con sus dependencias, lista para copiar. Un contenedor es una copia de la imagen en marcha.

Ciclo de vida

`docker run` lo crea y lo arranca, `docker stop` lo detiene y `docker rm` lo borra.

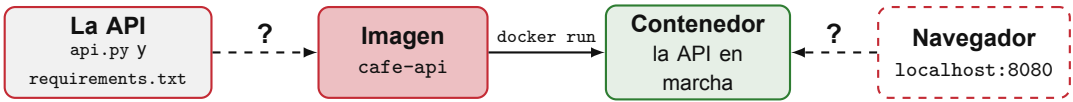
Responde

1. Diego copió `api.py` a su laptop y salió `ModuleNotFoundError`. ¿Qué le faltaba?

2. Lucía le pasa a Diego la **imagen** de la API. ¿Qué necesita Diego para correrla?

3. Detuviste `web` con `docker stop`. ¿Con qué comando lo ves todavía? ¿Cómo lo borras?

Revisión: la pregunta de hoy



1. Dockerfile	2. Puertos y variables	3. Compose
¿Cómo se arma una imagen propia con la API de Café Nube?	¿Cómo le llega un pedido al contenedor? ¿Cómo se configura sin cambiar la imagen?	Si la app tiene varias piezas, ¿cómo se levantan todas juntas?

Logro de la sesión

Al finalizar, habrás construido la **imagen** de la API de Café Nube con un **Dockerfile**, la habrás corrido con su **puerto publicado** y con una **variable de entorno**, y la habrás levantado junto a otro servicio con **Docker Compose**. Podrás explicar qué hace cada línea del Dockerfile y con qué comandos se maneja una app de varias piezas.

Saberes previos

- En la Sesión 27, ¿qué tuviste que instalar en tu computadora antes de encender la API de Café Nube?
- Cuando encendiste la API por segunda vez en el 8080, ¿qué pasó? ¿Cómo lo resolviste sin cambiar el código?
- ¿Qué hace `docker run hello-world` si la imagen no está en tu computadora?

Utilidad de esta sesión

Hasta ahora usaste imágenes que hizo otro: `hello-world` y `nginx`. Hoy armarás una **propia**, con la API de Café Nube: así corre igual en tu laptop, en la de Diego o en un servidor, **sin instalar Python ni Flask**. Y cuando una app tiene varias piezas, **Compose** las levanta todas con un comando.

Práctica

La API de Café Nube en una imagen, con Docker en tu computadora

Práctica: los archivos

Caso de estudio

La API de **Café Nube** de la Sesión 27 necesitaba Python, Flask y un puerto libre. Hoy va dentro de una **imagen**: quien la quiera correr solo necesita **Docker**.

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["python", "api.py"]
```

compose.yaml

```
services:
  api:
    build: .
    ports:
      - "8080:8080"
  web:
    image: nginx:1.29
    ports:
      - "8081:80"
```

Archivo	Qué es
`api.py`	La API de la Sesión 27
`requirements.txt`	`flask==3.1.0`
`Dockerfile`	Los pasos para armar la imagen
`compose.yaml`	La API y un Nginx, juntos

Un cambio en `api.py`: `app.run(host="0.0.0.0", ...)`, para aceptar pedidos que llegan **de afuera** del contenedor.

Práctica: construir y correr

Windows (PowerShell), con Docker Desktop abierto

```
# 1. Construye la imagen
docker build -t cafe-api .
# 2. Córrela con su puerto publicado
docker run -d --name api -p 8080:8080 cafe-api
# 3. Pídele la carta
curl.exe localhost:8080/menu
# 4. Lee sus mensajes
docker logs api
```

Parte	Qué hace
`docker build`	Arma la imagen con el `Dockerfile`
`-t cafe-api`	Le pone nombre a la imagen
`.`	La carpeta actual, donde está el `Dockerfile`
`-p 8080:8080`	Tu 8080 lleva al 8080 del contenedor

Comprueba en el navegador: `http://localhost:8080/menu`

```
[{"nombre":"Americano","precio":8},{"nombre":"Capuchino","precio":10},
 {"nombre":"Mocaccino","precio":12}]
```

Carpeta `Sesion28/practica_cafe_nube/`. La guía completa está en el `README.md`.

Práctica: seis pruebas

#	Qué hacer	Qué observar
1	`docker build -t cafe-api .` y `docker images cafe-api`	¿Cuántos pasos muestra? ¿Encendió la API?
2	Córrela con `-p 8080:8080` y pide `/menu`	¿Qué responde? ¿Qué anota `docker logs api`?
3	Otra copia, `api2`, con `-p 8080:8080`; después, con `-p 8081:8080`	¿Qué error sale? ¿Por qué con el 8081 sí?
4	`api3` con `-e PUERTO=9090 -p 8082:9090`	¿En qué puerto escucha adentro? Mira `docker ps`
5	Agrega un café a `MENU` y corre otra vez `api`; después, construye otra vez	¿Aparece? ¿Qué pasos dicen `CACHED`?
6	`docker compose up -d`, `ps`, `logs api` y `down`	¿Cuántos contenedores levanta? Abre `localhost:8081`

La idea

La imagen trae Python, Flask y el código: basta Docker. El puerto se abre con `-p` y la configuración va con `-e`. Reto en el `README.md`: `/menu/<nombre>` con Compose.

Docker desde cero y Compose

Dockerfile, puertos, variables de entorno y varias piezas

Dockerfile: tu propia imagen

Dockerfile

Un archivo de texto, llamado exactamente `Dockerfile`, con **los pasos para armar una imagen**. Cada línea empieza con una **instrucción** en mayúsculas.

Proyecto `hola-docker`: Dockerfile

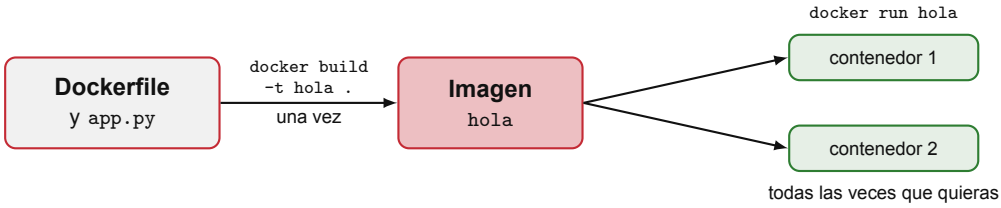
```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

app.py

```
import platform
print("Hola desde un contenedor")
print("Este programa corre en:",
      platform.system())
```

Instrucción	Qué hace
`FROM`	Elige la **imagen base**, de la que partes: `python:3.12-slim` ya trae Python 3.12
`WORKDIR`	La carpeta, dentro de la imagen, donde pasa todo lo que sigue
`COPY`	`COPY origen destino`: copia un archivo de tu proyecto a la imagen. El `.` es la carpeta de `WORKDIR`
`CMD`	El comando que corre **cuando arranca** un contenedor: `python app.py`, escrito como lista

Construir y correr




```
$ docker build -t hola .
 => [1/3] FROM docker.io/library/python:3.12-slim
 => [2/3] WORKDIR /app
 => [3/3] COPY app.py .
 => naming to docker.io/library/hola
$ docker run hola
Hola desde un contenedor
Este programa corre en: Linux
```

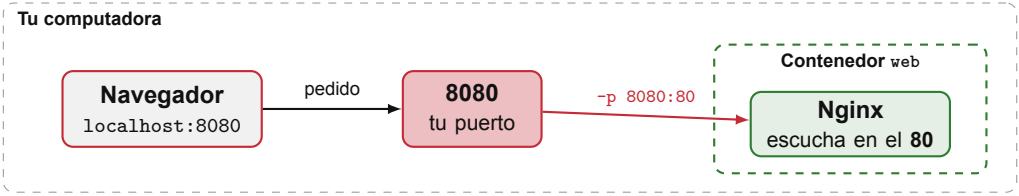
Construir (*build*)

Armar una imagen siguiendo el Dockerfile **de arriba hacia abajo**. Aunque uses Windows o Mac, dentro del contenedor es **Linux**.

Ojo

`CMD` no corre al construir: corre al arrancar. Y si cambias `app.py`, **construye otra vez**.

Puertos: publicar un puerto



Puerto

Un número que dice **a qué programa** de una computadora va un pedido. Nginx escucha en el 80; la API de Café Nube, en el 8080.

Publicar un puerto

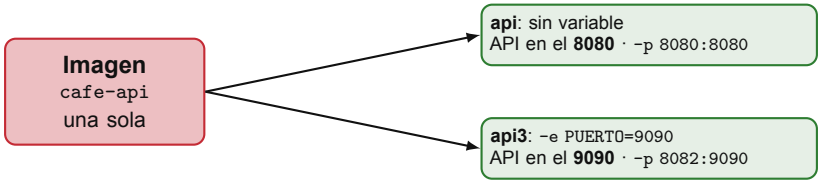
`-p tu-puerto:puerto-del-contenedor`. El contenedor está **aislado**: sin `-p`, nada de afuera le llega.

```
docker run -d --name web -p 8080:80 nginx
curl localhost:8080
docker rm -f web    # detiene y borra
```

port is already allocated

Dos contenedores no pueden usar el mismo puerto de **tu computadora**. El del contenedor sí se repite: `-p 8081:80`.

Variables de entorno



Variable de entorno	Para qué sirve
Un valor con nombre, como `PUERTO=9090`, que el programa lee **al arrancar**. Se le pasa al contenedor con `-e NOMBRE=valor`: un `-e` por cada una.	Lo que cambia de un lugar a otro (un puerto, una contraseña, el nombre de la tienda) no va dentro de la imagen: **la misma imagen** sirve en todos lados.

Producción

El lugar donde corre la app que usan los clientes de verdad. Usa la misma imagen que tu computadora, con su propia configuración.

Una base de datos que pide contraseña

Café Nube va a guardar los pedidos en una **base de datos**: un programa que guarda información ordenada y la entrega cuando se la piden. **PostgreSQL** tiene su imagen oficial, `postgres`, y **no arranca sin contraseña**.

✗ **Sin la variable**

```
$ docker run -d --name db postgres:17
$ docker logs db
Error: Database is uninitialized and
  superuser password is not specified.
  You must specify POSTGRES_PASSWORD
  to a non-empty value for the superuser.
```

El contenedor se detiene enseguida: sus **logs** dicen por qué.

✓ **Con la variable**

```
$ docker run -d --name db \
    -e POSTGRES_PASSWORD=secreto postgres:17
$ docker logs db
... database system is ready to accept
    connections
$ docker rm -f db
```

`:17` elige la versión. PostgreSQL escucha en el puerto **5432**.

Cuidado

Una contraseña real no se escribe en el Dockerfile ni en el código: va en una variable de entorno, al correr el contenedor.

Café Nube: la API en una imagen

Enunciado

La API de la carta solo corre en la laptop de Lucía. Hay que empaquetarla para que corra igual en cualquier computadora: debe **partir de Python 3.12**, **instalar las librerías de** `requirements.txt`, **responder en el puerto 8080** y quedar funcionando **sin pasos extra**.

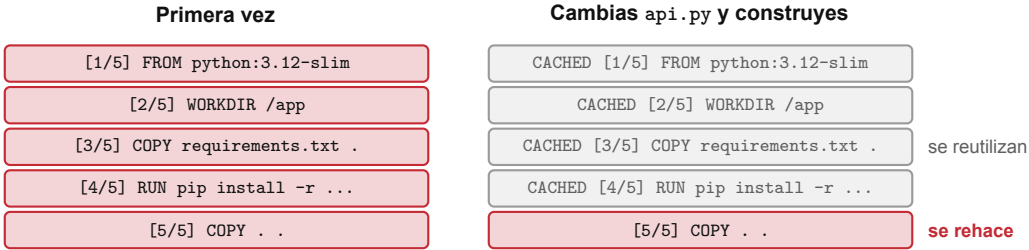
```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["python", "api.py"]
```

El enunciado dice…	…y se vuelve
partir de Python 3.12	`FROM python:3.12-slim`
instalar las librerías	`COPY requirements.txt` y `RUN pip install`
(el código de la API)	`COPY . .`
responder en el 8080	`EXPOSE 8080`, y al correrla `-p 8080:8080`
sin pasos extra	`CMD ["python", "api.py"]`

RUN: ejecuta un comando **mientras se construye** la imagen. Aquí instala Flask.

EXPOSE: anota en qué puerto escucha la app. Solo lo documenta: el puerto lo abre `-p`.

Capas: por qué importa el orden



Capa

Lo que deja cada línea del Dockerfile, guardado aparte. Si una línea y las de arriba no cambiaron, Docker **reutiliza su capa** (`CACHED`) en vez de rehacerla.

Por eso, este orden

Primero `requirements.txt` y `pip install`; **después** `COPY . .`. Un cambio en `api.py` no vuelve a instalar Flask: lo viste en la prueba 5.

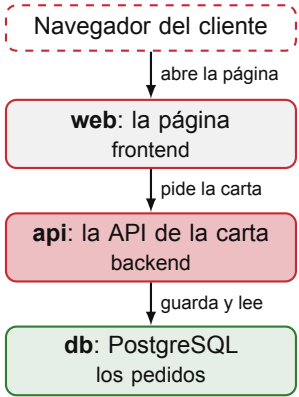
Café Nube: errores frecuentes

Lo que ves	Por qué y cómo se arregla
`No module named 'flask'` en `docker logs api`	Faltó instalar las librerías: agrega `RUN pip install -r requirements.txt` y construye otra vez
`curl` no conecta	Faltó `-p 8080:8080` al correrla, o el contenedor se detuvo: mira `docker logs api`
`The container name "/api" is already in use`	Ya hay un contenedor `api`: bórralo con `docker rm -f api`
`port is already allocated`	Otro contenedor usa ese puerto de tu computadora: elige otro, como `-p 8081:8080`
El cambio en `api.py` no aparece	La imagen tiene la copia vieja: `docker build -t cafe-api .` otra vez

Lo primero que miras

Cuando un contenedor falla o se detiene solo, `docker logs` dice por qué.

Una aplicación, varias piezas



Cada pieza, en su propio contenedor.

Frontend y backend

El **frontend** es lo que se ve en el navegador: la página de la carta. El **backend** es la API que le entrega los datos.

Servicio

Cada pieza de la app, con su imagen y su configuración. Café Nube tiene **tres**: `web`, `api` y `db`.

Un programa por contenedor

Así cambias la página sin tocar la API, y reinicias la API sin apagar la base.

Docker Compose: un archivo, un comando

✗ Con `docker run`

```
docker build -t cafe-api .
docker run -d --name api -p 8080:8080 cafe-api
docker run -d --name web -p 8081:80 nginx:1.29
```

Un comando largo por pieza, en orden, y hay que recordarlos cada vez. Con la base de datos, uno más.

✓ Con Docker Compose

```
services:
  api:
    build: .
    ports:
      - "8080:8080"
  web:
    image: nginx:1.29
    ports:
      - "8081:80"
```

Todo en `compose.yaml`, y un comando: `docker compose up -d`.

Docker Compose

Una herramienta de Docker que lee los servicios de una app desde un archivo, `compose.yaml`, y **los levanta todos con un solo comando**. El archivo va junto al código: quien lo descarga levanta la app igual que tú.

Viene con Docker Desktop: `docker compose version`. Se escribe `docker compose`, **con espacio**; `docker-compose`, con guion, es la versión antigua.

El archivo `compose.yaml`

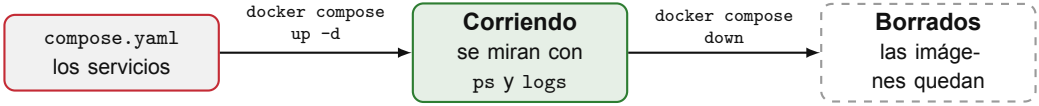
YAML

Un formato de texto para configuración: `nombre: valor` en cada línea. **La sangría dice qué va dentro de qué**, y una lista se escribe con guion: `- elemento`.

```yaml
services:
  api:
    build: .
    ports:
      - "8080:8080"
  web:
    image: nginx:1.29
    ports:
      - "8081:80"
```

Clave	Qué dice	Con `docker`
`services:`	La lista de servicios, cada uno con dos espacios de sangría	—
`build: .`	Construye la imagen con el `Dockerfile` de esta carpeta	`docker build .`
`image: nginx:1.29`	Usa una imagen ya hecha, sin construir nada	`docker run nginx:1.29`
`ports:`	Los puertos publicados, uno por línea	`-p 8081:80`

Compose: up, ps, logs y down



Con docker	Con docker compose	Qué hace
`docker run -d, uno por uno`	`docker compose up -d`	Construye lo que falte y levanta todo
`docker ps`	`docker compose ps`	Lista los contenedores del proyecto
`docker logs api`	`docker compose logs api`	Los mensajes de un servicio
`docker rm -f, uno por uno`	`docker compose down`	Detiene y borra todos los contenedores

Cambiaste el código

`up -d` reutiliza la imagen que ya existe. Para que construya otra vez: `docker compose up -d --build`.

Cierre: preguntas para comprobar

Responde con tus palabras

1. En la prueba 1, `docker build` no encendió la API. ¿Qué línea del Dockerfile la enciende y cuándo corre?

2. ¿Por qué pudiste correr `api` y `api2` a la vez, si los dos escuchan en el 8080?

3. En `-e PUERTO=9090 -p 8082:9090`, ¿qué número es de tu computadora y cuál del contenedor?

4. Agregaste un café a `api.py`. ¿Qué hiciste para verlo en el contenedor? ¿Por qué no se volvió a instalar Flask?

5. Después de `docker compose down`, ¿qué queda en tu computadora: los contenedores o las imágenes?

Idea final

El **Dockerfile** lleva los pasos para armar la imagen; `-p` abre el camino hasta el contenedor y `-e` le da su configuración sin cambiar la imagen. Cuando la app tiene varias piezas, **Compose** las describe en un archivo y las maneja con un comando.

Referencias

- Práctica: `github.com/rgap/rgap-Integracion-de-Servicios-Cloud-Gcp-Agosto2026`, carpeta `Sesion28/practica_cafe_nube/`.
- Ruta *Despliegue con Docker*: curso 1, *Docker desde cero* (Dockerfile, puertos, variables de entorno y el escenario Café Nube), y curso 2, *Varios contenedores con Compose* (lecciones 1 y 2).
- Docker, Dockerfile y Docker Compose: `docs.docker.com`. Imágenes `python`, `nginx` y `postgres`: `hub.docker.com`.
- Material previo del curso: Sesión 27 (desplegar, imagen, contenedor y su ciclo de vida, y la API de Café Nube).
- Las respuestas de las preguntas están en `../respuestas.md`.